Très bonne question Loïc — et c'est exactement le genre de détail qui fait la différence entre *"je fais marcher une API"* et *"je comprends vraiment comment un serveur web fonctionne"*.
On va décortiquer ça proprement.

🎯 Pourquoi le DocumentRoot pointe vers public/ ?

Ce n'est pas un hasard.
C'est un **choix d'architecture volontaire**, utilisé dans tous les frameworks modernes :

- Laravel → `public/`
- Symfony → `public/`
- CodeIgniter → `public/`
- Slim → `public/`
- Vue/React/Angular → `dist/` (équivalent)

Pourquoi ?
Parce que **DocumentRoot = ce que le navigateur peut voir**.

Donc si tu mets ton DocumentRoot dans `public/`, tu contrôles **exactement** ce qui est accessible depuis l'extérieur.

🔥 1. Apache ne peut servir que ce qui est dans le DocumentRoot

Quand tu appelles :

```
http://localhost/api/public/tasks
```

Apache traduit ça en :

```
C:/wamp64/www/api/public/tasks
```

Donc **tout ce qui est en dehors de** `public/` **est invisible** pour le navigateur.

C'est une protection énorme.

🔥 2. Pourquoi ne pas mettre DocumentRoot sur `/api/` directement ?

Parce que si tu fais ça, Apache exposerait :

```
/src/config.php
/src/database.php
/src/TaskController.php
```

➡️ Et ça, c'est catastrophique.

Imagine que quelqu'un puisse accéder à :

```
http://localhost/api/src/config.php
```

Il verrait :

- ton mot de passe MySQL
- ton nom de base
- ton utilisateur
- ta logique interne

C'est un **risque de sécurité majeur**.

🔥 3. Le dossier `public/` sert de "zone sûre"

Dans `public/`, tu mets uniquement :

- `.htaccess`
- `index.php`
- des assets (CSS, JS, images)

Tout le reste (contrôleurs, modèles, config, classes) reste **hors de portée**.

C'est exactement comme si tu avais :

```
api/
├── public/        ← visible par Apache
```

```
└── src/            ← invisible, sécurisé
```

🔥 4. Le rôle du `.htaccess` dans `public/`

Le `.htaccess` dit à Apache :

"Si l'URL ne correspond pas à un fichier réel,
envoie tout vers index.php."

C'est ce qui permet d'avoir des routes REST comme :

```
/tasks
/tasks/1
/tasks/1/edit
```

Même si ces fichiers n'existent pas physiquement.

🔥 5. Pourquoi `public/` est le meilleur choix pour une API REST ?

Parce que :

✅ Sécurité

Tu caches ton code sensible.

✅ Propreté

Tu sépares :

- le point d'entrée HTTP (`public/`)
- la logique métier (`src/`)

✅ Standard moderne

Tous les frameworks sérieux utilisent cette structure.

✅ Facilité de déploiement

Sur un serveur, tu dis simplement :

```
DocumentRoot /var/www/api/public
```

Et tu es sûr que rien d'autre n'est exposé.

🎯 Résumé simple

Apache sait que `public/` est le DocumentRoot parce que **tu le lui dis** dans la configuration (ou Wamp le déduit du dossier où tu mets ton projet).

Et on utilise `public/` comme DocumentRoot parce que :

- c'est **sécurisé**
- c'est **propre**
- c'est **standard**
- ça permet de **contrôler ce qui est exposé**
- ça permet à `.htaccess` de router toutes les requêtes vers `index.php`

Si tu veux, je peux t'expliquer :

- comment Apache choisit le DocumentRoot dans Wamp
- comment créer un VirtualHost propre pour ton API
- comment sécuriser encore plus ton architecture

Tu veux aller dans quelle direction maintenant ?